

Universidad Autónoma de Santo Domingo

PRIMADA DE AMÉRICA / Fundada el 28 de octubre de 1538

Presentación

Facultad

Ciencias

Escuela

Escuela de informática

Asignatura

Lab Lenguaje de Programación III (INF 5170)

Tema

Resumen de JSON y su uso

Estudiante

Cristopher Santiago Helena Perez

Matrícula

100658882

Profesor/a

Radhames Silverio Gónzalez

Fecha

1/11/2026

Índice

Introducción 3

1. ¿Qué es JSON? 4

2. ¿Cuál es el propósito de JSON? 4

3. Características principales de JSON 4

4. Ventajas y desventajas del uso de JSON 5

Ventajas: 5

Desventajas: 5

5. Diferencia entre JSON y otros formatos de datos 6

6. Usos actuales de JSON en la industria 6

7. Ejemplo de un documento JSON y explicación de sus partes 7

Explicación de sus partes principales: 7

Conclusión 8

Referencias Bibliográficas 8

Introducción

El imparable avance del desarrollo web moderno y la proliferación de aplicaciones interactivas del lado del cliente —como las denominadas *Single Page Applications* (SPA) y las aplicaciones móviles multiplataforma— transformaron radicalmente las exigencias de comunicación en la arquitectura de redes. Durante años, los sistemas dependieron de estructuras rígidas y pesadas para transmitir datos a través de Internet. Sin embargo, con la evolución de la Web 2.0 y la necesidad imperiosa de lograr una sincronización de datos instantánea, fluida y con el menor consumo posible de recursos de red y procesamiento, se consolidó la urgencia de adoptar un estándar de intercambio de información más ágil, minimalista y directamente compatible con los lenguajes de programación del navegador.

Es en este contexto evolutivo donde emerge con fuerza revolucionaria **JSON** (*JavaScript Object Notation*). Concebido originalmente a principios de la década del 2000 por Douglas Crockford como un subconjunto del lenguaje de programación JavaScript, JSON fue diseñado bajo la premisa de ofrecer un formato de texto completamente independiente de cualquier lenguaje, pero sumamente intuitivo para los desarrolladores. Su filosofía minimalista eliminó la verbosidad de las etiquetas de cierre tradicionales, permitiendo representar objetos complejos y arreglos de datos mediante una sintaxis limpia basada en pares de clave y valor. Hoy en día, JSON se erige como el formato universal e indiscutible para la comunicación en APIs RESTful, servicios web y arquitecturas de microservicios distribuidos a escala global.

Informe sobre JSON

1. ¿Qué es JSON?

JSON (*JavaScript Object Notation* o Notación de Objetos de JavaScript) es un formato de texto ligero, estructurado, autónomo y completamente independiente del lenguaje de programación. Aunque su sintaxis está directamente inspirada en la forma en que los objetos y arreglos literales son declarados en JavaScript, JSON es un estándar agnóstico que cuenta con bibliotecas, analizadores (*parsers*) y generadores nativos o de terceros en prácticamente todos los entornos de desarrollo modernos (Python, Java, C#, PHP, C++, Go, entre otros). Un archivo o flujo de datos JSON es esencialmente una cadena de caracteres codificada habitualmente en UTF-8, lo que garantiza su legibilidad y transmisión universal sin problemas de compatibilidad de caracteres.

2. ¿Cuál es el propósito de JSON?

El propósito fundamental de JSON es servir como un mecanismo eficiente, estandarizado y universal para el **intercambio de datos estructurados entre sistemas informáticos heterogéneos**, facilitando principalmente la comunicación asíncrona entre un servidor backend y una aplicación cliente (ya sea un navegador web, una app móvil o un software de escritorio). Sus objetivos clave incluyen:

- **Serialización rápida de objetos:** Convertir estructuras de datos complejas en memoria (como clases o arreglos de objetos) en un formato plano de texto que pueda viajar a través de una red HTTP.
- **Eliminación de la complejidad de análisis:** Permitir que los lenguajes interpreten los datos entrantes de manera directa, transformándolos en estructuras nativas de forma inmediata y con un mínimo uso de ciclos de CPU.
- **Desacoplamiento de arquitecturas:** Permitir que aplicaciones desarrolladas en tecnologías completamente diferentes se entiendan mediante un protocolo de datos común.

3. Características principales de JSON

Estructura basada en Clave-Valor y Colecciones: Organiza la información mediante dos estructuras universales: un objeto (colección desordenada de pares clave/valor delimitados por llaves {}) y un arreglo o lista (colección ordenada de valores delimitados por corchetes []).

Tipos de datos primitivos y complejos soportados:

- Cadenas de texto (*Strings*) entre comillas dobles.
- Números (enteros o de punto flotante).

- Booleanos (`true` o `false`).
- Nulos (`null`).
- Objetos anidados y Arreglos.

Ausencia de etiquetas de cierre: A diferencia de XML, no utiliza marcas redundantes de apertura y cierre para cada elemento, lo que reduce drásticamente el peso del archivo.

Inexistencia de código ejecutable: A pesar de derivar de la sintaxis de JavaScript, JSON es estrictamente un formato de datos y **no contiene funciones, métodos, bucles ni código ejecutable**, lo que lo hace seguro frente a inyecciones de código malicioso al momento de ser interpretado.

4. Ventajas y desventajas del uso de JSON

Ventajas:

- **Extremadamente Ligero y Eficiente:** Al prescindir de etiquetas repetitivas, el tamaño del archivo se reduce al mínimo, optimizando notablemente el ancho de banda y acelerando la velocidad de transferencia en redes móviles o de alta latencia.
- **Compatibilidad Nativa con JavaScript:** Los navegadores web interpretan JSON de manera instantánea utilizando métodos integrados optimizados por el motor de JavaScript (`JSON.parse()` y `JSON.stringify()`), sin requerir librerías externas.
- **Legibilidad Humana y Machine-friendly:** Su estructura jerárquica basada en indentación es fácil de leer para los programadores y sumamente sencilla de procesar mediante algoritmos informáticos.
- **Amplio Ecosistema de Soporte:** Es el estándar nativo de la inmensa mayoría de las APIs modernas, bases de datos NoSQL (como MongoDB) y sistemas de almacenamiento en la nube.

Desventajas:

- **Incapacidad para incluir comentarios:** La especificación oficial de JSON prohíbe explícitamente el uso de comentarios (`//` o `/* ... */`), lo que dificulta documentar archivos de configuración extensos directamente dentro del código.
- **Ausencia de un esquema de validación nativo:** A diferencia de XML (que cuenta con DTD y XSD integrados), JSON puro no valida de manera nativa los tipos de datos ni las restricciones de estructura, requiriendo herramientas externas complementarias como *JSON Schema*.
- **Limitaciones para tipos de datos complejos:** No soporta de forma nativa tipos de datos avanzados como fechas (*Date*), expresiones regulares (*RegExp*), funciones o referencias circulares, los cuales deben convertirse obligatoriamente a cadenas de texto antes de ser serializados.

5. Diferencia entre JSON y otros formatos de datos

- **Sintaxis y Verbocidad:** XML utiliza una estructura de etiquetas angulares explícitas de apertura y cierre (<etiqueta>valor</etiqueta>), resultando verboso y pesado. JSON utiliza una notación compacta basada en llaves, corchetes y pares clave-valor ("clave": "valor").
- **Rendimiento y Peso en Red:** JSON es considerablemente más ligero y rápido de analizar (*parsear*) por los navegadores y servidores, consumiendo menos memoria RAM y CPU en comparación con XML.
- **Manejo de Metadatos y Atributos:** XML permite definir atributos dentro de las etiquetas de apertura y metadatos complejos. JSON no posee el concepto de atributos; toda la información se modela estrictamente como propiedades de objetos o elementos de arreglos.
- **Validación:** XML posee mecanismos de validación de esquemas robustos y maduros incorporados históricamente en la industria empresarial (XSD). JSON requiere validadores de terceros basados en especificaciones como *JSON Schema*.

6. Usos actuales de JSON en la industria

- **APIs RESTful y Web Services Modernos:** Es el formato estándar absoluto para el intercambio de peticiones y respuestas HTTP en arquitecturas web y microservicios.
- **Aplicaciones Móviles y Web Dinámicas (*SPAs*):** Utilizado por frameworks frontend como React, Angular y Vue.js, así como por aplicaciones nativas en iOS y Android para sincronizar datos en tiempo real.
- **Archivos de Configuración de Software:** Empleado masivamente en herramientas de desarrollo modernas (por ejemplo, package.json en Node.js, configuraciones en editores de código y entornos de despliegue).
- **Bases de Datos NoSQL (Orientadas a Documentos):** Motores de almacenamiento de alta escalabilidad como MongoDB y CouchDB almacenan sus registros de manera nativa utilizando variantes binarias optimizadas de JSON (*BSON*).

7. Ejemplo de un documento JSON y explicación de sus partes

```JSON
{
  "empresa": "AbbaTech",
  "activo": true,
  "anioFundacion": 2026,
  "sucursales": ["Santo Domingo", "Santiago", "Punta Cana"],
  "productoDestacado": {
    "codigo": "PROD-99",
    "nombre": "Laptop Gamer Pro",
    "precio": 1250.00,
    "especificaciones": {
      "procesador": "Intel Core i7",
      "memoriaRAM": "32GB",
      "almacenamiento": "1TB SSD"
    },
    "disponible": true
  }
}
```

Explicación de sus partes principales:

1. **Llaves Principales ({ }):** Delimitan los límites del objeto JSON raíz que contiene toda la estructura de datos del documento.

2. **Pares Clave-Valor ("empresa": "AbbaTech"):** Elementos fundamentales donde la **clave** (siempre encerrada entre comillas dobles) actúa como identificador y el **valor** representa el dato asignado.

3. **Tipos de Datos Primitivos:**

4. Cadenas de texto (*Strings*) como "AbbaTech" o "Santo Domingo".

5. Booleanos (true) que indican estados lógicos.

6. Numéricos (1250.00) sin comillas, permitiendo operaciones matemáticas directas.

7. **Arreglos o Listas ([ ]):** Secuencias ordenadas de valores agrupados entre corchetes, como el listado de sucursales ["Santo Domingo", "Santiago", "Punta Cana"].

8. **Objetos Anidados ("productoDestacado": {...} y "especificaciones": {... }):** Objetos dentro de otros objetos que permiten modelar relaciones jerárquicas complejas y multidimensionales de forma limpia y ordenada.

Conclusión

En conclusión, el formato **JSON** (*JavaScript Object Notation*) ha revolucionado de manera definitiva la forma en que los sistemas informáticos estructuran, transportan y consumen información en la web contemporánea. Al ofrecer una alternativa extraordinariamente ligera, rápida de procesar e intuitiva frente a los modelos tradicionales basados en XML, JSON se consolidó como el motor invisible que sostiene la comunicación instantánea en las aplicaciones móviles, las interfaces de usuario interactivas y las arquitecturas de servicios distribuidos. Su perfecta sinergia con los lenguajes de programación modernos y su adopción masiva como el estándar indiscutible en las APIs RESTful garantizan que su relevancia y vigencia tecnológica perdurarán como un pilar fundamental en el desarrollo de software actual y futuro.

Referencias Bibliográficas

1. **Crockford, D.** (2006). *The application of JSON for data interchange*. RFC 4627. Internet Engineering Task Force (IETF). Recuperado el 31 de octubre de 2026, de https://www.ietf.org/rfc/rfc4627.txt

2. **Mozilla Developer Network (MDN).** (2025). *Working with JSON*. Mozilla Contributors. Recuperado el 31 de octubre de 2026, de https://developer.mozilla.org/es/docs/Learn/JavaScript/Objects/JSON

3. **ECMA International.** (2017). *The JSON Data Interchange Standard* (ECMA-404, 2nd ed.). Standard ECMA. Recuperado el 31 de octubre de 2026, de https://www.ecma-international.org/publications-and-standards/standards/ecma-404/

4. **W3Schools.** (s. f.). *JSON Introduction*. Refsnes Data. Recuperado el 31 de octubre de 2026, de https://www.w3schools.com/js/js_json_intro.asp